

DRV8214 Ripple-Counting & Stall Detection — Test and Calibration Plan

Össur i-limb ultra (medium) · ESP32-S3 · 5 V / 0.8 A derated scheme

Revision: v1.0 **Baseline references:** DRV8214 datasheet SLVSH04 (Nov 2023) §8.3.4-8.3.7, §8.6, §9.2.2-9.2.3; Nayyar thesis (KMC/KMC_SCALE characterisation, inrush, LCR resistance); i-limb clinician manual (nominal kinematics).

0. Executive summary — read this before writing any DoE code

There is a **hard inconsistency in the inherited baseline that must be resolved first**. The ~32,000 ripples/stroke figure from the KMC = 5 / KMC_SCALE = 11b (S4) configuration cannot be commutation ripples. Three independent arguments:

Argument 1 — the device's own speed ceiling. The DRV8214 represents ripple speed as $\text{SPEED} \times \text{W_SCALE}$ (rad/s). The maximum representable value, at $\text{W_SCALE} = 11\text{b}$, is 32,640 rad/s:

$$f_{\text{ripple,max}} = \frac{32,640}{2\pi} = 5,195 \text{ Hz}$$

At 5 V the predicted stroke time is 1.18-1.46 s (derived in §2). 32,000 ripples over that window implies:

$$f_{\text{ripple}} = \frac{32,000}{1.2 \dots 1.46} = 22,000 \dots 27,000 \text{ Hz}$$

That is **4-5× beyond the maximum ripple rate the DRV8214 can represent at all**. The speed estimator that sets the band-pass centre frequency physically cannot be tracking that signal.

Argument 2 — PWM aliasing. 27 kHz down to 22 kHz brackets the 25 kHz internal PWM frequency ($\text{PWM_FREQ} = 0\text{b}$) to within a couple of percent. The most probable explanation is that at S4/K5 the band-pass filter's centre frequency estimate has collapsed and the counter is locking onto **PWM switching residue on the current sense node**, not brush commutation. This is fully consistent with the thesis observation that *"anything below KMC = 5 read noise"* — KMC = 5 sits directly on the noise cliff, and PWM noise is periodic, so it produces a **low coefficient of variation**. Repeatable noise is still noise. The thesis correctly selected S4/K5 on a repeatability criterion, but repeatability was never cross-checked against absolute accuracy.

Argument 3 — motor kinematics. $32,000 \div 14 \text{ ripples/rev} = 2,286 \text{ motor revolutions per stroke}$. Over 1.3 s that is $\approx 98,000 \text{ rpm}$. No 6-8 mm coreless motor in this class exceeds $\sim 20,000 \text{ rpm}$, and at 5 V with 0.8 A regulation the available back-EMF is only 2.92 V (§2).

Expected true count. Bounding by the device ceiling and by realistic motor speed:

$$N_{\text{total,max}} = f_{\text{ripple,max}} \times t_{\text{stroke}} = 5,195 \times 1.46 = 7,585$$

Combined with a plausible 8,000-16,000 rpm operating band, **expect the true full-stroke count to land in the 2,000-7,500 range**.

Gate condition: Do not begin the filter DoE (§4) until Test 0 has established a ground-truth ripple frequency. Every DoE response metric — CV, drift, asymmetry — will be beautifully optimised on the wrong signal otherwise. This is the single highest-value hour of bench time in the whole programme.

1. Test 0 — Ground-truth accuracy audit (blocking prerequisite)

1.1 Objective

Establish f_{ripple} and N_{total} by a measurement path that does not depend on the DRV8214's own filter, then re-tune KMC/KMC_SCALE against it using the datasheet's §9.2.3.1.2.2 procedure.

1.2 Method A — oscilloscope on IPROPI (the datasheet's recommended path)

1. Isolate one motor. Drive it unladen at 5 V / 0.8 A, mid-stroke, at steady state.
2. Scope `IPROPI` (AC-coupled, 50 mV/div, 20 MHz BW limit on). Datasheet requires ≥ 20 **ripples** in the capture window for a valid frequency estimate.
3. Take an FFT. You will see two families:
 - a sharp line at 25 kHz (or 50 kHz) plus harmonics → **PWM, ignore**
 - a broader peak in the 1–5 kHz region that **shifts with load** → **commutation, this is your signal** The load-shift test is the discriminator: apply a light finger load and confirm the peak moves down while the PWM line does not.
4. Record $f_{\text{ripple,free}}$ (unladen) and $f_{\text{ripple,loaded}}$ (at 0.8 A regulation). These two numbers feed §3 (FLT_K) directly.

Convert for the datasheet's tuning flow:

$$\omega_{\text{ripple}} = 2\pi f_{\text{ripple}} \quad [\text{rad/s}] \quad (\text{Eq. 11})$$

1.3 Method B — firmware-only cross-check (no scope)

The speed estimator derives ω from back-EMF, i.e. from $(V_M - I_M \cdot R_m)$ — an **electrically independent path** from the band-pass counter. Compare the two:

$$\rho = \frac{N_{\text{total}}/t_{\text{stroke}}}{\text{SPEED} \times \text{W_SCALE}/2\pi}$$

- $\rho \approx 1.0 \pm 0.1$ → counter and estimator agree; the configuration is self-consistent.
- $\rho \gg 1$ → **over-counting** (the current suspicion; expect $\rho \approx 5$).
- $\rho \ll 1$ → under-counting / dropped ripples.

If `SPEED` saturates at 0xFF, W_SCALE is too small; if `SPEED` reads 0x00–0x03 the estimator has lost lock. Log both raw `SPEED` and ρ as first-class telemetry channels throughout the whole programme.

1.4 Method C — voltage invariance (strongest single test, needs a bench supply)

N_{total} is a **purely kinematic quantity**: gear ratio $\times$ mechanical travel. It is independent of supply voltage. Only the *frequency* scales. Therefore:

$$N_{\text{total}}(5 \text{ V}) \equiv N_{\text{total}}(7.4 \text{ V}) \quad \text{within measurement error}$$

Run 10 strokes at 5 V and 10 at 7.4 V (current limit unchanged). If the counts differ by more than ~3 %, the counter is voltage-sensitive, which means it is tracking an electrical artefact rather than mechanics. **Mark this as a permanent regression test** — re-run it after any register change.

1.5 KMC / KMC_SCALE re-tune

With ω_{tuned} from Method A, use the datasheet's proportionality method (Eq. 16) rather than the from-scratch binary search — it converges in one step:

$$\frac{\omega_{\text{tuned}}}{\omega_{\text{def}}} = \frac{\text{KMC_SCALE}_{\text{tuned}}}{\text{KMC}_{\text{tuned}}} \times \frac{\text{KMC}_{\text{def}}}{\text{KMC_SCALE}_{\text{def}}}$$

with $\text{KMC}_{\text{def}} = 163$, $\text{KMC_SCALE}_{\text{def}} = 11\text{b} = 2^4 \times 2^{13}$. Obtain ω_{def} by writing the defaults, running a stroke, and reading $\boxed{\text{SPEED} \times \text{W_SCALE}}$. Then choose $\text{KMC_SCALE}_{\text{tuned}}$ from the four options such that $\text{KMC}_{\text{tuned}}$ lands in 0-255 with the highest precision.

Sanity target: with $f_{\text{ripple}} \approx 3 \text{ kHz}$, $\omega \approx 18,850 \text{ rad/s}$, set $\boxed{\text{W_SCALE} = 11\text{b}}$ (max 32,640 rad/s) and expect $\boxed{\text{SPEED} \approx 147}$.

1.6 Fixed parameters to lock before anything else

Parameter	Value	Derivation
$\boxed{\text{INV_R_SCALE}}$	$\boxed{01\text{b}}$ (64)	Only workable scale — see below
$\boxed{\text{INV_R}}$	25	$64/2.6 = 24.6 \rightarrow 25$
$\boxed{\text{FLT_GAIN_SEL}}$	$\boxed{11\text{b}}$ (16)	Datasheet: use full signal range
$\boxed{\text{VSNS_SEL}}$	$\boxed{0\text{b}}$	Analog output filter (recommended)
$\boxed{\text{W_SCALE}}$	$\boxed{11\text{b}}$ (128)	Headroom to 32,640 rad/s
$\boxed{\text{PWM_FREQ}}$	$\boxed{0\text{b}}$ (25 kHz)	Hold constant; it is a confound

INV_R quantisation warning. The candidate scales give: $2/2.6 = 0.77 \rightarrow 1$ (useless); $64/2.6 = 24.6 \rightarrow 25$; $1024/2.6 = 394$ (> 255); $8192/2.6 = 3151$ (> 255). So $\boxed{01\text{b}}/25$ is forced, and the represented resistance is $64/25 = 2.56 \Omega$ — a -1.5 % error, with ~4 % **per LSB** resolution around 2.6 Ω . At 0.8 A this is a 32 mV error on a 2.92 V back-EMF estimate, i.e. ~1.1 % **speed bias**. Tolerable, but it is a hard floor on estimator accuracy and it means you *cannot* thermally compensate $\boxed{\text{INV_R}}$ in fine steps (see §6, edge case 4).

2. Derated voltage & frequency calibration

2.1 Scaling the manufacturer spec

Steady-state armature equation:

$$V_{\text{term}} = I \cdot R_m + K_e \omega \quad \Rightarrow \quad \omega = \frac{V_{\text{term}} - I R_m}{K_e}$$

Since stroke time $t \propto 1/\omega$ and K_e is invariant:

$$t_{5V} = t_{\text{nom}} \times \frac{V_{\text{nom}} - IR_m}{V_{\text{test}} - IR_m}$$

With $t_{\text{nom}} = 0.8 \text{ s}$, $V_{\text{nom}} = 7.4 \text{ V}$, $V_{\text{test}} = 5.0 \text{ V}$, $R_m = 2.6 \Omega$:

Regime	Current used	Predicted t_{5V}	Comment
Naive (ignores IR)	—	1.18 s	Lower bound only; do not use
Unladen	$I_0 \approx 0.20 \text{ A}$	1.23 s	Free-air stroke
Current-limited	$I_{\text{lim}} = 0.80 \text{ A}$	1.46 s	Loaded / regulating

Expect your measured value in the 1.2–1.5 s window. Which end it lands on is diagnostic: near 1.23 s means the finger never reaches the 0.8 A limit during a free stroke; near 1.46 s means the driver is regulating for most of the stroke (check `IMTR` and `IN_DUTY` to confirm).

Available back-EMF at 5 V — this number drives §3 and §4:

$$E_{\text{free}} = 5 - 0.20(2.6) = 4.48 \text{ V} \quad E_{\text{loaded}} = 5 - 0.80(2.6) = 2.92 \text{ V}$$

2.2 Derated ripple frequency

$$f_{\text{ripple}} = \frac{N_R \cdot n_{\text{motor}}}{60} = \frac{N_{\text{total}}}{t_{\text{stroke}}} \quad N_R = \text{LCM}(N_B, N_C) = \text{LCM}(2, 7) = 14$$

Scaling from a known nominal frequency:

$$f_{\text{ripple},5V} = f_{\text{ripple},7.4V} \times \frac{V_{\text{test}} - IR_m}{V_{\text{nom}} - IR_m}$$

2.3 Firmware routine T1 — automated stroke characterisation

Configuration for this test:

- `EN_STALL = 1b`, `S.MODE = 0b` (latched disable — the endstop is the timing marker and the motor must not cook)
- `STALL_REP = 1b` so nFAULT falls on stall → capture the edge in a GPIO ISR, **not** by I²C polling. I²C polling adds up to one full poll period of timing error.
- `TINRUSH`: 22 ms measured inrush + 35 % margin = 30 ms → $30 \text{ ms} / 102.4 \mu\text{s} = 293 = 0x0125$
- `EN_SS = 0b` (soft-start OFF for characterisation — with `EN_SS = 1b` the datasheet requires `tINRUSH = TINRUSH × WSET_VSET`, which changes the meaning of the register)
- `IMODE = 00b` (no current regulation) so the stall comparator is unambiguous — see §5.1

cpp

```
// ---- Test T1: full-stroke characterisation -----
struct StrokeResult {
    uint32_t t_stroke_us;
    uint16_t rc_cnt;
    uint8_t speed_mean;
    uint8_t fault;
    bool timed_out;
};

volatile uint32_t g_nfault_us[N_MOTORS] = {0};

void IRAM_ATTR isr_nfault_m0() { g_nfault_us[0] = (uint32_t)esp_timer_get_time(

StrokeResult run_stroke(uint8_t m, bool close_dir, uint32_t timeout_us = 3'000'
    StrokeResult r{};
    drv_write(m, REG_CONFIG0, cfg0 | BIT_CLR_CNT | BIT_CLR_FLT); // zero counter
    drv_write(m, REG_CONFIG0, cfg0); // release self
    g_nfault_us[m] = 0;

    uint32_t t0 = (uint32_t)esp_timer_get_time();
    drv_set_direction(m, close_dir); // I2C_EN_IN1 /
    drv_enable(m, true);

    uint32_t sum_speed = 0, n_speed = 0;
    while (!g_nfault_us[m]) {
        if ((uint32_t)esp_timer_get_time() - t0 > timeout_us) { r.timed_out = true;
            sum_speed += drv_read(m, REG_RC_STATUS1); n_speed++; // SPEED, oppor
            vTaskDelay(pdMS_TO_TICKS(2));
        }
        drv_enable(m, false);

        r.t_stroke_us = (g_nfault_us[m] ? g_nfault_us[m] : (uint32_t)esp_timer_get_ti
        r.rc_cnt = drv_read16(m, REG_RC_STATUS2); // RC_CNT[15:0]
        r.speed_mean = n_speed ? (uint8_t)(sum_speed / n_speed) : 0;
        r.fault = drv_read(m, REG_FAULT); // classify STA
        return r;
    }
}
```

Protocol: $N = 20$ strokes per direction. **Enforce thermal discipline:** 3 s idle between strokes, 60 s every 10 strokes. Copper winding resistance rises $\sim 0.4\text{ \%/}^{\circ}\text{C}$, so back-to-back strokes will show a monotonic drift in t_{stroke} that will alias onto any factor you sweep monotonically. Log the trial index and the elapsed-since-cold time as columns so you can regress the drift out.

Report: mean, SD, CV of t_{stroke} ; mean N_{total} ; derived f_{ripple} ; the consistency ratio ρ from §1.3.

2.4 RC_THR sizing (do this *after* the count is trusted)

$$N_{RT} = \text{RC_THR} \times \text{RC_THR_SCALE}, \quad \text{RC_THR} \in [0, 1023], \text{RC_THR_SCALE} \in \{2, 8, 16\}$$

Set $N_{RT} \approx 0.95 \times N_{\text{total}}$ so `CNT_DONE` latches just before the mechanical endstop — this gives you a soft-landing trigger to ramp speed down before contact.

- If $N_{\text{total}} \approx 32,000$: `RC_THR_SCALE = 11b (64)`, `RC_THR = 475` → 30,400
- If $N_{\text{total}} \approx 4,000$: `RC_THR_SCALE = 01b (8)`, `RC_THR = 475` → 3,800

3. Inertia-based seeding of FLT_K and T_MECH_FLT

3.1 FLT_K — band-pass Q from the speed envelope

`FLT_K` is the 1/Q factor, so:

$$\text{BW} = \frac{f_c}{Q} = f_c \times \text{FLT_K} \quad \Rightarrow \quad \text{FLT_K}_{\min} = \frac{f_{\max} - f_{\min}}{f_c}$$

Ripple frequency is proportional to back-EMF, so the full operating envelope at 5 V is $E_{\text{loaded}} : E_{\text{free}} = 2.92 : 4.48$. Placing f_c at the geometric mean:

$$\text{FLT_K}_{\min} = \sqrt{\frac{4.48}{2.92}} - \sqrt{\frac{2.92}{4.48}} = 1.239 - 0.807 = \mathbf{0.432}$$

Nearest code: `0110b` = **0.5 — the datasheet default**. That is a useful result: the 5 V / 0.8 A envelope happens to demand almost exactly the default bandwidth, so the default is a principled starting point here rather than a lazy one.

Note the filter **re-centres continuously** on the estimated speed, so FLT_K only needs to cover the estimator's *tracking error*, not the full static range. That argues for sweeping *below* 0.5 as well.

DoE levels: `0100b (0.125)`, `0101b (0.25)`, `0110b (0.5)`, `1000b (0.75)`. Note that codes `1010b`–`1111b` all evaluate to 1 — do not waste runs there.

Trade-off to expect: wider BW → tracks fast speed changes, fewer missed ripples during transients, but admits more brush-bounce and PWM sideband energy → over-count. Narrower BW → cleaner, but drops ripples whenever the speed slews faster than the estimator can follow (exactly what happens on object contact).

3.2 T_MECH_FLT — output LPF from the measured inertia

The datasheet gives **no numeric cutoff table** for `T_MECH_FLT`. You must build that table for your system. Fortunately you already have an inertia proxy: the **22 ms measured inrush duration**.

Inrush current decays as back-EMF builds, $i(t) = i_{\text{stall}}e^{-t/\tau_m} + i_{ss}$. Taking 22 ms as visual settling ($\approx 3.5\tau$):

$$\tau_{\text{mech}} \approx \frac{22 \text{ ms}}{3.5} = 6.3 \text{ ms} \quad f_{\text{mech}} = \frac{1}{2\pi\tau_{\text{mech}}} \approx \mathbf{25 \text{ Hz}}$$

Design window for the ripple-counter output LPF:

$$3f_{\text{mech}} \leq f_{\text{LPF}} \leq \frac{f_{\text{ripple}}}{10} \quad \Rightarrow \quad 75 \text{ Hz} \leq f_{\text{LPF}} \leq 300 \text{ Hz}$$

(lower bound: don't attenuate genuine speed transients; upper bound: actually do some smoothing). Roughly a decade wide — consistent with the datasheet's "default `100b` suffices."

Firmware routine T2 — step-response characterisation. This builds the missing datasheet table:

1. Put the driver in voltage regulation ($\text{REG_CTRL} = 11\text{b}$), ($\text{EN_SS} = 0\text{b}$).
2. Command a step in (WSET_VSET) from 50 % to 100 % mid-stroke (use a long-travel jig or a de-clutched motor so you don't hit an endstop).
3. Log (SPEED) at ≥ 500 Hz through the transient.
4. Fit the 10–90 % rise time t_r .
5. Sweep (T_MECH_FLT) = 0...7 and plot t_r vs code.

The cascade of mechanical and filter poles combines as $t_r \approx \sqrt{t_{r,\text{mech}}^2 + t_{r,\text{flt}}^2}$. The intercept at code 0 gives $t_{r,\text{mech}}$; the growth gives $t_{r,\text{flt}}(\text{code})$, i.e. your empirical cutoff table.

Selection rule: choose the **largest** code for which $\tau_{\text{flt}} \leq \tau_{\text{mech}}/3$. Larger codes buy noise immunity for free until the filter starts dominating the mechanical response, at which point stall-detection latency grows.

DoE levels: (010b), (100b) (default), (101b), (110b).

4. Design of Experiments

4.1 Held-constant factors (do not vary these mid-DoE)

($\text{EN_RC}=1\text{b}$), ($\text{FLT_GAIN_SEL}=11\text{b}$), ($\text{VSNS_SEL}=0\text{b}$), (PWM_FREQ), (W_SCALE), (INV_R), (INV_R_SCALE), (KMC), (KMC_SCALE) (post-§1.5), (TINRUSH), supply voltage, (RIPROPI), and (CS_GAIN_SEL) **above all**.

(CS_GAIN_SEL) is a triple-confound. It lives in (RC_CTRL0 (0x11)) alongside (EN_RC), (DIS_EC) and (FLT_GAIN_SEL), and it simultaneously sets (a) the current-mirror gain feeding the ripple detector, (b) the (ITRIP) stall threshold via Eq. 3, and (c) the OCP trip point. Changing it moves three responses at once and invalidates the entire DoE block. Fix it in §5.2 and never touch it again.

4.2 Blocked / nuisance factors

Motor temperature (block by cool-down + trial index), direction (block: run both, analyse separately), driver/finger identity (block: characterise the index finger fully, then spot-check the other four), supply rail sag (log (VMTR) every sample), mechanical wear (re-run a fixed reference configuration every 100 strokes as a control chart).

Randomise run order. Do not sweep (FLT_K) monotonically — thermal drift will alias directly onto it and you will "discover" a beautiful, entirely spurious main effect.

4.3 Response variables

#	Metric	Formula	Needs ground truth?
R1	Absolute error	$\varepsilon = \bar{N} - N_{\text{truth}} /N_{\text{truth}}$	Yes (§1)
R2	Repeatability	$CV = \sigma/\mu$ over 10 reps	No
R3	Directional asymmetry	$A = \bar{N}_{cl} - \bar{N}_{op} /\frac{1}{2}(\bar{N}_{cl} + \bar{N}_{op})$	No
R4	Cyclic closure drift	$D = P_K /K$ ripples/cycle	No
R5	Load sensitivity	$\Delta N = \bar{N}_{\text{load}} - \bar{N}_{\text{free}} /\bar{N}_{\text{free}}$	No
R6	False stall rate	events per 50 strokes	No

R4 is the primary objective and deserves explanation. Run $K = 10$ close/open cycles *without* clearing `RC_CNT` between them, accumulating a signed position estimate $P = \sum(\pm \Delta \text{RC_CNT})$. After an even number of cycles the finger is physically back where it started, so P must equal zero. Any residual is pure accumulated counting error. Convert to something physical:

$$D_{\text{mm}} = D \times \frac{\text{stroke}_{\text{mm}}}{N_{\text{total}}}$$

This gives you a ground-truth-free accuracy metric in millimetres of fingertip travel per cycle — exactly the number that matters for a PID position loop, and the number a clinician would recognise.

R3 matters specifically here because the thesis already observed a close/open discrepancy. Under a rigid, backlash-bounded gear train both directions traverse identical distance; a large A is direct evidence of *load-dependent* counting error, since closing and opening differ mainly in the spring/tendon load profile.

Objective function: minimise D (R4), subject to $CV < 2\%$ (R2), $\varepsilon < 5\%$ (R1), and $R6 = 0$.

4.4 Stage A — filter screening with the error corrector OFF

`DIS_EC = 1b`.

This is the most important structural decision in the DoE. With error correction enabled, the corrector **synthesises missing pulses and suppresses extra ones**. A band-pass filter that is dropping 30 % of ripples will still produce a smooth, repeatable, entirely plausible count. You will measure the corrector, not the filter, and you will select the wrong `FLT_K`. Characterise the raw detector first, always.

- Design: **full factorial 4×4** (`FLT_K` $\times$ `T_MECH_FLT`) = 16 cells
- Reps: 10 per cell per direction = 320 strokes
- Runtime: ≈ 4 s/cycle including cool-down $\rightarrow \approx$ **22 min**
- Analysis: two-way ANOVA on $\log(CV)$ and $\log(D)$, with interaction; block on direction and thermal batch.
- Carry forward the **top 2** cells.

4.5 Stage B — error corrector tuning, EC ON

`DIS_EC = 0b`, `FLT_K`/`T_MECH_FLT` fixed at the Stage A winners.

- Design: full factorial `EC_FALSE_PER` (4) × `EC_MISS_PER` (4) = 16 cells, at `EC_PULSE_DIS = 1b` (see §5.3 for why 1b is mandatory)
- Run each cell under **three load conditions**: free air, constant light load (soft foam block), transient load (rigid object introduced mid-stroke at a fixed position via a jig)
- $16 \times 3 \times 10$ reps = 480 strokes $\approx$ 35 min
- Primary responses here: R4 (drift) and R5 (load sensitivity)
- Then a separate 2-cell confirmation of `EC_PULSE_DIS` = 0b/1b on the disconnected-motor and hard-stall edge cases only

4.6 Stage C — confirmation

Winner configuration, 50 strokes per direction per finger, all five motors, plus the §1.4 voltage-invariance regression test. Freeze as `CONFIG_V1` in firmware with a version hash embedded in every telemetry file header.

5. Dynamic load and stall detection strategy

5.1 The core problem: ITRIP and current regulation share one VREF

The stall comparator and the current regulator **both** compare `V_IPROPI` against `V_VREF`. There is one VREF pin per device. Therefore **you cannot set a regulation limit of 0.8 A and a stall threshold above it on the same driver**. If you regulate at 0.8 A, the regulator pins the current at precisely the level that would declare a stall — the two are definitionally ambiguous.

This is almost certainly the root cause of the thesis finding that the IPROPI signal "*often did not change at all during stalls*". It was not a sensitivity problem; the regulator was clamping the very signal being used for detection.

Two resolutions:

(a) Electrical detection — drop current regulation. Set `IMODE = 00b`. Size `VREF` for 0.8 A. On stall, current rises from the running value, crosses `ITRIP` at 0.8 A, and `SMODE = 0b` latches the outputs off within $t_{\text{BLANK}} + t_{\text{DEG}} \approx 4 \mu\text{s}$. Peak current never meaningfully exceeds 0.8 A (unregulated stall current would be $5/2.6 = 1.92$ A, but the latch fires first). Clean and unambiguous.

(b) Kinematic detection — motor powered, ripples stopped. Independent of current entirely, and the more robust criterion for a prosthetic because it catches compliant/soft stalls where current never spikes sharply.

Recommendation: dual-criterion, with kinematic as primary.

5.2 IPROPI chain sizing (fix this before Stage A)

Datasheet Eq. 3: $I_{\text{TRIP}} \times A_{\text{IPROPI}} = V_{\text{VREF}} / R_{\text{IPROPI}}$

The thesis firmware uses `IPROPI_RESISTOR = 4700`. Check the combinations:

CS_GAIN_SEL	A_{IPROPI}	OCP min	$V_{IPROPI} @ 0.8 \text{ A}, 4.7 \text{ k}\Omega$	Verdict
010b	1125 $\mu\text{A/A}$	800 mA	4.23 V	✗ Exceeds the 3.3 V clamp <i>and</i> the VM-1.25 V headroom rule. Also puts OCP exactly on ITRIP.
000b	225 $\mu\text{A/A}$	4 A	0.846 V	✓ Valid, but uses only 26 % of a 3.3 V ADC range

Action: use **CS_GAIN_SEL = 000b** (225 $\mu\text{A/A}$, OCP 4 A) and **raise RIPROPI to ~11 k Ω :**

- $V_{IPROPI} @ 0.8 \text{ A} = \mathbf{1.98 \text{ V}}$ — good ADC utilisation, headroom to ~1.3 A before the 3.3 V clamp
- V_{VREF} for $I_{TRIP} = 0.8 \text{ A} \rightarrow \mathbf{1.98 \text{ V}}$ (set externally; **INT_VREF = 0b**)
- Satisfies $V_{VREF} \leq V_{VM} - 1.25 = 3.75 \text{ V}$ ✓

Note that **INT_VREF = 1b** (500 mV fixed) would give $I_{TRIP} = 0.5 / (225\mu \times 11\text{k}) = 0.20 \text{ A}$ — far too low. Use external VREF.

5.3 EC_PULSE_DIS is mandatory if you use kinematic stall detection

From datasheet Table 8-19: with **DIS_EC = 0b** and **EC_PULSE_DIS = 0b**, **RC_OUT** continues to output pulses even when the motor is disconnected, not rotating, or stalled.

This completely defeats ripple-rate stall detection — **RC_CNT** keeps climbing during a stall, so the ripple rate never drops and the detector never fires. **You must set EC_PULSE_DIS = 1b**, which halts pulses once the corrector has added 12 consecutive synthetic pulses with no band-pass output.

Two consequences to design around:

1. ****Detection latency.**** The stall is only visible after 12 synthetic ripple periods:

$$t_{\text{lat,EC}} = \frac{12}{f_{\text{est}}}$$

At $f = 3 \text{ kHz} \rightarrow 4 \text{ ms}$ (negligible). But f_{est} *falls* as the motor decelerates into the stall, so the window stretches. At $f = 300 \text{ Hz} \rightarrow 40 \text{ ms}$. ****This is the dominant and least obvious latency term in the whole detection chain — measure it explicitly rather than assuming.****

2. **Position error injection.** Those 12 synthetic pulses are added to **RC_CNT**. Every contact event silently injects up to 12 phantom ripples. Over a day of grasping this accumulates. Mitigate by subtracting 12 on each **STALL** assertion, and — far better — by **homing**: every full-open stroke terminates at a hard mechanical endstop, so reset the position estimate to zero there. Homing bounds drift to one stroke's worth regardless of how bad the corrector is.

5.4 Physically-derived priors for EC_FALSE_PER / EC_MISS_PER

Both windows are expressed as a percentage of the expected inter-ripple interval $T_e = 1/f_{\text{est}}$. The right settings follow from how much that interval can genuinely change between consecutive ripples:

$$\frac{\Delta T_e}{T_e} \approx \frac{T_e}{\tau} = \frac{1}{f_{\text{ripple}} \cdot \tau}$$

Scenario	τ	f	Fractional interval change
Free acceleration / load release	6.3 ms	3 kHz	5.3 %
Rigid object contact	~1 ms	3 kHz	33 %

EC_FALSE_PER (**blanking window — rejects extra ripples**). A real ripple would have to arrive at < 20 % of the expected interval to be wrongly discarded, i.e. the speed would need to nearly quintuple between two adjacent ripples. At 5.3 % max acceleration, **this constraint never binds**. So **EC_FALSE_PER** should be set purely from the *noise* side, to reject brush-bounce doublets and PWM sidebands. Given the strong over-count suspicion in §0, **start aggressive: 11b (50 %)** and only reduce if you see systematic under-count.

EC_MISS_PER (**insertion window — synthesises missing ripples**). This one *does* bind. On rigid contact the interval can genuinely stretch by ~33 % as the motor decelerates. If **EC_MISS_PER = 00b** (20 %), the corrector will insert a phantom ripple on every real deceleration → **systematic over-count under load, and a masked stall**. Set it **above** the contact-deceleration figure: **10b (40 %) baseline, sweep 10b/11b**.

The prior is therefore **deliberately asymmetric**: tight on false-positive rejection, loose on missed-ripple insertion. That is the correct bias for a prosthetic, where load transients are the norm and a masked stall is a safety issue.

5.5 Ripple-rate stall detector

Sample **RC_CNT** at $T_s = 5$ ms. Ripple rate $r_k = (\text{RC_CNT}_k - \text{RC_CNT}_{k-1})/T_s$.

Declare **STALL_KIN** when $r_k < \alpha \cdot f_{\text{ripple,free}}$ for M consecutive samples, with $\alpha \approx 0.10$ -0.15.

Confirmation window: $t_{\text{confirm}} = MT_s$. Bound it:

- **Lower:** $t_{\text{confirm}} \geq 5\tau_{\text{mech}} = 32$ ms, so a transient load dip cannot trigger it
- **Upper:** keep it small relative to stroke time and below the crush threshold

Target $t_{\text{confirm}} = 40$ ms ($M = 8$) — about 3 % of a 1.3 s stroke.

Total worst-case latency budget:

$$t_{\text{total}} = t_{\text{lat,EC}} + t_{\text{confirm}} + t_{\text{I2C}} \approx 40 + 40 + 5 = 85 \text{ ms}$$

Validate this against a force gauge: the peak force applied to a compliant object between contact and motor shutdown must stay under your safety budget.

cpp

```
// ---- Dual-criterion stall detector (per motor) -----
typedef enum { ST_IDLE, ST_INRUSH, ST_RUN, ST_STALL_PEND, ST_STALLED } stall_state_t;

typedef struct {
    stall_state_t st;
    uint16_t last_cnt;
    uint32_t last_us;
    uint8_t low_rate_streak;
    float r_thresh; // ripples/s, = ALPHA * f_ripple_free
    uint32_t drive_start_us;
} stall_ctx_t;

#define ALPHA 0.12f
#define M_CONFIRM 8 // x 5 ms = 40 ms
#define T_INRUSH_US 30000 // must match TINRUSH register

void stall_update(stall_ctx_t *c, uint16_t rc_cnt, uint8_t fault_reg, uint32_t now_us)
// Electrical criterion: hardware latch already acted (SMODE=0b). Trust it implicitly.
if (fault_reg & FAULT_STALL) { c->st = ST_STALLED; return; }
if (fault_reg & FAULT_OCP) { c->st = ST_STALLED; return; } // classify severe faults as stalls

if (c->st == ST_INRUSH) {
    if (now_us - c->drive_start_us < T_INRUSH_US) { c->last_cnt = rc_cnt; c->last_us = now_us;
    c->st = ST_RUN; // blanking expires
    }
    if (c->st != ST_RUN && c->st != ST_STALL_PEND) return;

    uint32_t dt = now_us - c->last_us;
    if (dt < 3000) return; // enforce >=3ms
    uint16_t d = rc_cnt - c->last_cnt; // unsigned wrap-around
    float r = (float)d * 1e6f / (float)dt;
    c->last_cnt = rc_cnt; c->last_us = now_us;

    if (r < c->r_thresh) {
        if (++c->low_rate_streak >= M_CONFIRM) { c->st = ST_STALLED; }
        else { c->st = ST_STALL_PEND; }
    } else {
        c->low_rate_streak = 0;
        c->st = ST_RUN;
    }
}
}
```

Note the `d = rc_cnt - c->last_cnt` unsigned subtraction handles the 16-bit `RC_CNT` wrap correctly without a branch.

6. Edge cases — false stall vs true mechanical stall

1. Direction reversal transient. Reversing at speed produces a far larger current spike than a cold start, because the back-EMF *adds* to the supply:

$$I_{\text{rev}} = \frac{V_{VM} + K_e \omega}{R_m} = \frac{5 + 2.92}{2.6} = 3.05 \text{ A}$$

That is $\sim 4\times$ your `ITRIP` and, if `CS_GAIN_SEL` were left at `010b`, above the 800 mA OCP limit too. **Never command a direct reversal.** Enforce brake (`LL`) for $\geq 3 \tau_{\text{mech}} \approx 20 \text{ ms}$, or enable `EN_SS` soft-start in a regulation mode. Add this as a hard interlock in the motion FSM, not a convention.

2. OCP masquerading as stall. Both `OCP` and `STALL` pull `nFAULT` low. With `OCP_MODE = 1b` the device auto-retries every 1.7 ms, producing a chatter pattern that superficially resembles repeated stall detection. Always read the `FAULT` register (0x00) to distinguish `STALL`/`OCP`/`TSD`/`OVP` — never infer the cause from the pin alone. Sizing `CS_GAIN_SEL = 000b` (OCP 4 A) puts OCP a factor of 5 above `ITRIP` and makes the distinction unambiguous.

3. I²C timeout impersonating a stall. If a transaction NACKs or times out and your driver layer returns the last-known value, `RC_CNT` plateaus — **indistinguishable from a mechanical stall in the kinematic detector.** Check the `Wire.endTransmission()` return code on every transaction, maintain a per-motor bus-error counter, stream it as a telemetry channel, and gate `stall_update()` on a successful read. This is the failure mode most likely to cause a spurious mid-grasp shutdown in the field.

4. Thermal drift of R_m. `INV_R` is set from a cold 2.6 Ω. A 100 °C rise takes the true value to $\sim 3.0 \Omega$. The back-EMF estimate ($V_M - IR_m$) then over-estimates speed $\rightarrow$ the band-pass centre frequency runs high $\rightarrow$ missed ripples $\rightarrow$ progressive under-count. **Test:** 30 consecutive strokes, plot N_{total} vs trial index; a monotonic trend is the signature. Mitigation: set `INV_R` for a mid-range *warm* resistance rather than cold. Note that the $\sim 4\%$ /LSB quantisation (§1.6) means you can only compensate in coarse steps — the practical options around 2.6 Ω are `INV_R` = 26 (2.46 Ω), 25 (2.56 Ω), 24 (2.67 Ω).

5. Soft stall vs hard stall. A hard endstop or rigid object produces a sharp current step (electrical criterion fires first). A compliant object — foam, a paper cup, skin — produces gradual speed decay with little current step, and **only the kinematic criterion will catch it.** This is precisely why the dual-criterion design is not redundancy for its own sake.

6. Stall position as a diagnostic. The datasheet's own gas-valve example applies directly: *the number of ripples before stall should be identical for each actuation.* Therefore:

- Stall at $RC_CNT \approx N_{\text{total}} \rightarrow$ **normal endstop**, home the position here
- Stall at $RC_CNT \ll N_{\text{total}}$ and no commanded grasp $\rightarrow$ **unexpected obstruction or mechanical jam**, raise a distinct fault
- Stall at a *repeatable* intermediate count across many cycles $\rightarrow$ **mechanical fault** (tendon fray, gear damage, debris). Log the histogram of stall positions; it is a free predictive-maintenance signal.

7. `RC_CNT` 16-bit rollover. Max 65,535. If $N_{\text{total}} \approx 32,000$, two strokes without `CLR_CNT` overflow. Either clear per stroke, or configure `RC_REP` for the desired max-value behaviour and handle the wrap in a 32-bit software accumulator. Do not leave this to chance.

8. Back-driving while idle. If a finger is pushed by an external force with outputs in Hi-Z, the motor back-drives and generates real commutation ripples that the driver will not count (no current through the sensed low-side FETs). The position estimate silently corrupts with

no fault flag. **Set idle state to brake** (`LL`), **not coast**. The current mirror senses in both drive and brake low-side slow-decay periods, so braking both resists back-driving *and* keeps the ripple counter alive. Also investigate the `RC_HIZ` bit in `RC_CTRL0` for the Hi-Z counting case.

9. Five-motor rail interaction. Simultaneous starts draw up to $5 \times 0.8 \text{ A} = 4 \text{ A}$ from the 5 V rail. Sag corrupts *all five* back-EMF estimates at once, so the resulting count errors will be correlated and may look like a systematic effect rather than a supply problem. Stagger motor starts by $\sim 10 \text{ ms}$, size `CBULK` accordingly (datasheet example uses $50 \mu\text{F}$ per driver), and log `VMTR` (`REG_STATUS1`) on every sample so you can correlate anomalies against rail voltage after the fact.

10. `CNT_DONE` is latched. It stays high until `CLR_CNT`. If you use it as a soft-landing trigger, clear it at the start of every stroke or the second stroke will trigger immediately.

7. Telemetry and data acquisition architecture

7.1 Bandwidth budget

I²C. The DRV8214 supports Fast Mode, `fSCL` max **400 kHz**. The thesis code runs at 100 kHz — **raise it to 400 kHz** (with $2.2 \text{ k}\Omega$ pull-ups as the datasheet specifies for the address pins; size bus pull-ups for your capacitance).

Per-motor telemetry read = 6 bytes (`0x00` FAULT, `0x01` SPEED, `0x02`–`0x03` RC_CNT, `0x04` VMTR, `0x05` IMTR).

Access mode	Per motor	5 motors	Headroom @ 200 Hz (5 ms)
Burst / auto-increment	$\sim 200 \mu\text{s}$	$\sim 1.0 \text{ ms}$	Comfortable
Six single-byte reads	$\sim 600 \mu\text{s}$	$\sim 3.0 \text{ ms}$	Tight

Verify burst reads empirically, early. The datasheet's read sequence (Figure 8-21) shows a NACK after a single data byte and **does not document auto-increment**. If burst does not work, 200 Hz across five motors is still achievable but leaves little margin — plan for that.

Verdict: 400 kHz I²C, 200 Hz for all five motors, 500 Hz for single-motor characterisation.

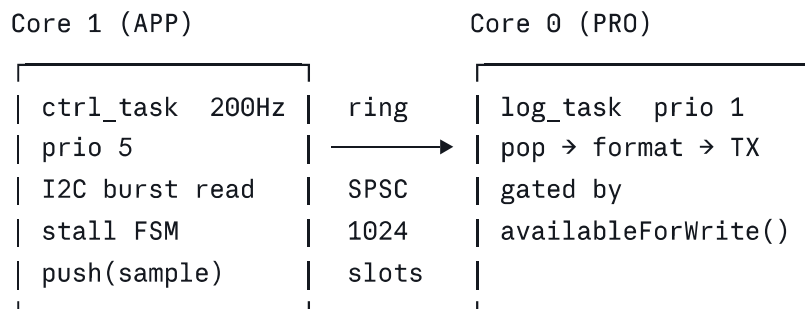
Serial. Use the ESP32-S3's **native USB-CDC port**, not the UART bridge. On native USB the baud parameter is nominal and real throughput is $\sim 600 \text{ kB/s}$ – 1 MB/s . If you must use the CP2102 bridge, 921600 is reliable and 2 Mbaud usually works.

Mode	Payload	Rate	Throughput
ASCII CSV	$\sim 40 \text{ B/row}$	$5 \times 200 \text{ Hz}$	$40 \text{ kB/s} \rightarrow \textbf{921600 baud minimum}$
Binary framed	12 B/sample	$5 \times 200 \text{ Hz}$	$12 \text{ kB/s} \rightarrow \text{comfortable}$

7.2 Architecture

Three rules, in priority order:

1. **Never call `Serial.print()` from the control task.** It blocks when the TX FIFO fills, and a blocked control task means missed I²C samples and a `RC_CNT` plateau — which your stall detector will read as a stall.
2. **Timestamp at the start of the I²C transaction,** not after formatting.
3. **Count and report dropped samples.** Silent loss in a DoE is worse than no data.



cpp

```
// ---- Telemetry sample & lock-free SPSC ring -----
typedef struct __attribute__((packed)) {
    uint32_t t_us;           // esp_timer_get_time(), taken pre-transaction
    uint16_t rc_cnt;         // 0x02..0x03
    uint8_t speed;           // 0x01
    uint8_t imtr;            // 0x05
    uint8_t vmtr;            // 0x04
    uint8_t fault;           // 0x00
    uint8_t motor_id;
    uint8_t state;           // stall FSM state / event tag
} sample_t;                // 12 bytes

#define RING_SZ 1024        // power of two
static sample_t s_ring[RING_SZ];
static volatile uint32_t s_head = 0, s_tail = 0;
static volatile uint32_t s_drops = 0;

static inline bool ring_push(const sample_t *s) {
    uint32_t h = s_head, n = (h + 1) & (RING_SZ - 1);
    if (n == s_tail) { s_drops++; return false; } // full: drop newest, keep co
    s_ring[h] = *s;
    __atomic_store_n(&s_head, n, __ATOMIC_RELEASE);
    return true;
}

static inline bool ring_pop(sample_t *out) {
    uint32_t t = s_tail;
    if (t == __atomic_load_n(&s_head, __ATOMIC_ACQUIRE)) return false;
    *out = s_ring[t];
    __atomic_store_n(&s_tail, (t + 1) & (RING_SZ - 1), __ATOMIC_RELEASE);
    return true;
}
```

cpp

```
// ---- Control task: fixed cadence, never blocks -----
static void ctrl_task(void *arg) {
    const TickType_t period = pdMS_TO_TICKS(5);    // 200 Hz
    TickType_t last = xTaskGetTickCount();
    for (;;) {
        for (uint8_t m = 0; m < N_MOTORS; m++) {
            sample_t s;
            s.t_us = (uint32_t)esp_timer_get_time(); // BEFORE the transaction
            uint8_t buf[6];
            if (!drv_read_burst(m, 0x00, buf, 6)) { // checks endTransmission()
                g_i2c_err[m]++;                    // do NOT feed the stall FSM
                continue;
            }
            s.fault = buf[0]; s.speed = buf[1];
            s.rc_cnt = (uint16_t)buf[2] | ((uint16_t)buf[3] << 8);
            s.vmtr = buf[4]; s.imtr = buf[5];
            s.motor_id = m;

            stall_update(&g_stall[m], s.rc_cnt, s.fault, s.t_us);
            s.state = (uint8_t)g_stall[m].st;
            ring_push(&s);
        }
        vTaskDelayUntil(&last, period);           // no cumulative drift
    }
}
```

cpp

```
// ---- Logger task: writes only what the TX buffer will accept -----
static char linebuf[128];

static void log_task(void *arg) {
    sample_t s;
    for (;;) {
        int budget = Serial.availableForWrite();
        while (budget > 96 && ring_pop(&s)) {
            int n;
            if (g_mode == MODE_PLOT) {
                // Arduino Serial Plotter: name:value pairs, ONE motor, decimated
                if (s.motor_id != g_plot_motor || (s.t_us / 10000) == g_last_plot_tick)
                    g_last_plot_tick = s.t_us / 10000; // ~100 Hz decimation
                n = snprintf(linebuf, sizeof(linebuf),
                    "rc:%u,spd:%u,i:%u,v:%u,st:%u\r\n",
                    s.rc_cnt, s.speed, s.imtr, s.vctr, s.state);
            } else { // MODE_LOG: full-rate CSV
                n = snprintf(linebuf, sizeof(linebuf),
                    "%lu,%u,%u,%u,%u,%u,0x%02X,%u\n",
                    (unsigned long)s.t_us, s.motor_id, s.rc_cnt, s.speed,
                    s.imtr, s.vctr, s.fault, s.state);
            }
            if (n <= 0 || n > budget) break;
            Serial.write((const uint8_t *)linebuf, n);
            budget -= n;
        }
        vTaskDelay(pdMS_TO_TICKS(2));
    }
}

void telemetry_begin() {
    Serial.setTxBufferSize(8192); // MUST precede begin()
    Serial.begin(921600);
    xTaskCreatePinnedToCore(ctrl_task, "ctrl", 4096, NULL, 5, NULL, 1);
    xTaskCreatePinnedToCore(log_task, "log", 4096, NULL, 1, NULL, 0);
}

```

7.3 CSV format — self-describing run files

Every run file must carry its own configuration. In a DoE the settings belong in the data, not in a lab notebook.

```
#RUN,id=A_017,fw=1.4.2+g8ac31f,utc=2026-07-30T14:22:10Z
#CFG,FLT_K=0x6,T_MECH_FLT=0x4,EC_FALSE_PER=0x3,EC_MISS_PER=0x2,DIS_EC=1,EC_PULS
#CFG,KMC=5,KMC_SCALE=0x3,INV_R=25,INV_R_SCALE=0x1,CS_GAIN_SEL=0x0,W_SCALE=0x3
#CFG,TINRUSH=293,SMODE=0,IMODE=0,VREF_mV=1980,RIPROPI=11000,VM_mV=5000
#COND,dir=close,load=free,trial=17,t_since_cold_s=124,motor=1
#COL,t_us,motor,rc_cnt,speed,imtr,vmtr,fault,state
1284310,1,0,0,0,0,0x00,1
1289315,1,47,142,38,161,0x00,2
...
#END,n_samples=1043,i2c_err=0,drops=0,t_stroke_us=1287420,rc_final=3912
```

The `#END` line closes the loop: `drops` and `i2c_err` must both be zero for the run to be admissible. Discard any run where they are not.

7.4 Serial Plotter vs real DAQ

The Arduino Serial Plotter is a **scope, not a DAQ**. It stalls above a few hundred lines/second, uses a fixed window, and IDE 2.x wants `name:value` pairs. It will also choke on a `#` header line.

- `MODE_PLOT`: one motor, ≤ 6 series, decimated to ~ 100 Hz, no header. For eyeballing a stroke shape or confirming a stall fires.
- `MODE_LOG`: full rate, all motors, full header, captured by a `pyserial` script to disk. This is what the DoE analysis consumes.

A minimal capture harness (host side) is worth 20 lines and saves hours:

```
python

import serial, sys, time
p = serial.Serial(sys.argv[1], 921600, timeout=1)
p.write(b"MODE LOG\nRUN A_017\n")
with open(f"run_{sys.argv[2]}.csv", "wb") as f:
    while True:
        line = p.readline()
        if not line: break
        f.write(line)
        if line.startswith(b"#END"): break
```

7.5 Optional independent channel — PCNT on RC_OUT

The ESP32-S3 has hardware pulse-counter units. `RC_OUT` is open-drain with $50\ \mu\text{s}$ positive pulses — well within PCNT's range at any plausible ripple rate. Wiring `RC_OUT` into PCNT gives you a **zero-CPU-cost count that is independent of the I²C read cadence**, so an I²C hiccup can no longer corrupt the position estimate.

Caveats: PCNT counters are 16-bit signed ($\pm 32,767$) — install an overflow ISR and accumulate in 32-bit software. There are only 4 units on the S3, so with 5 motors you will need to multiplex or accept coverage of 4. Note that `RC_OUT` is *post-error-correction*, so this is a redundancy channel, not an independent ground truth (that still requires the scope, §1.2).

8. Execution order and time budget

#	Activity	Prereq	Est. time
0	Rework <code>RIPROPI</code> → 11 kΩ, set <code>CS_GAIN_SEL=000b</code> , <code>VREF</code> =1.98 V; raise I²C to 400 kHz; verify burst reads	—	2 h
1	Test 0 — scope IPROPI, establish f_{ripple} , re-tune KMC/KMC_SCALE	0	3 h
2	Build telemetry stack (§7), validate zero drops at 200 Hz × 5	0	4 h
3	T1 — stroke-time & count characterisation, both directions, 20 reps	1, 2	1 h
4	Voltage-invariance regression (§1.4)	3	1 h
5	T2 — <code>T_MECH_FLT</code> step-response table	3	2 h
6	Stage A DoE — <code>FLT_K</code> × <code>T_MECH_FLT</code> , EC off	4, 5	1 h run + 2 h analysis
7	Stage B DoE — <code>EC_FALSE_PER</code> × <code>EC_MISS_PER</code> , 3 loads	6	1 h run + 2 h analysis
8	Stall latency validation vs force gauge	7	2 h
9	Stage C confirmation, all 5 motors; freeze <code>CONFIG_V1</code>	8	3 h

Total ≈ 24 h bench time. Steps 0 and 1 are the gate — nothing downstream is trustworthy until they close.

9. Register quick-reference (verify bit positions against §8.6 before shifting)

Addr	Name	Fields
0x00	FAULT	FAULT STALL OCP OVP TSD NPOR CNT_DONE (R)
0x01	RC_STATUS1	SPEED[7:0] (R)
0x02/0x03	RC_STATUS2/3	RC_CNT[7:0] / RC_CNT[15:8] (R)
0x04/0x05	REG_STATUS1/2	VMTR / IMTR (R)
0x09	CONFIG0	EN_OUT EN_OVP EN_STALL VSNS_SEL VM_GAIN_SEL CLR_CNT CLR_FLT DUTY_CTRL
0x0A/0x0B	CONFIG1/2	TINRUSH[7:0] / TINRUSH[15:8] — 102.4 µs/LSB
0x0C	CONFIG3	IMODE[7:6] SMODE[5] INT_VREF[4] TBLANK[3] TDEG[2] OCP_MODE[1] TSD_MODE[0]
0x0D	CONFIG4	RC_REP[7:6] STALL_REP[5] CBC_REP[4] PMODE[3] I2C_BC[2] I2C_EN_IN1[1] I2C_PH_IN2[0]
0x11	RC_CTRL0	EN_RC[7] DIS_EC[6] RC_HIZ[5] FLT_GAIN_SEL[4:3] CS_GAIN_SEL[2:0]
0x12	RC_CTRL1	RC_THR[7:0]
0x13	RC_CTRL2	INV_R_SCALE[7:6] KMC_SCALE[5:4] RC_THR_SCALE[3:2] RC_THR[9:8]
0x14/0x15	RC_CTRL3/4	INV_R[7:0] / KMC[7:0]
0x16	RC_CTRL5	FLT_K[7:4] (reset 6d)
0x17	RC_CTRL6	EC_PULSE_DIS[7] T_MECH_FLT[6:4] EC_FALSE_PER[3:2] EC_MISS_PER[1:0] (reset 45h)
0x18/0x19	RC_CTRL7/8	KP_DIV[7:5] KP[4:0] / KI_DIV[7:5] KI[4:0]

Registers marked (*) in the datasheet register map (REG_CTRL, PWM_FREQ, IMODE, SMODE, INT_VREF, TBLANK, TDEG, OCP_MODE, TSD_MODE, PMODE, I2C_BC, VSNS_SEL, VM_GAIN_SEL, DUTY_CTRL) are **writable only when** EN_OUT = 0. Your DoE runner must disable outputs before reconfiguring between cells, or the writes will silently no-op and you will run a whole block at the previous settings.